

Nexa

Um framework frontend sem nada para instalar.

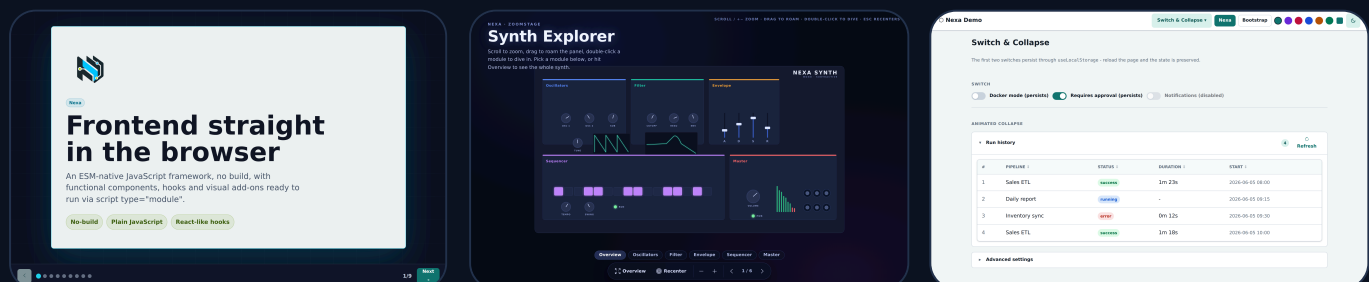
Sem build. Sem bundler. Sem node_modules. Nexa é um framework JavaScript no-build e ESM-nativo, com uma API de hooks parecida com React — o app inteiro são módulos ES que o navegador roda direto. Para uma revisão de segurança e arquitetura, essa única decisão é o que importa.

Zero dependências de runtime

Roda sob CSP estrita

~60 componentes + add-ons

MIT



Acima: três interfaces feitas com Nexa — uma apresentação com zoom, um painel de sintetizador e uma galeria de componentes. Todas servidas como arquivos estáticos, sem build.

01 Sumário executivo

O que é. Nexa é um framework de UI client-side cujos componentes são funções JavaScript simples e cujo runtime é um conjunto de módulos ES carregados com `<script type="module">`. Ele entrega a um time acostumado com React uma API de hooks familiar (`useState`, `useEffect`, `context`, `router`, `SSR`) e traz ~60 componentes de UI, um design system e add-ons visuais — sem nenhum compilador no meio.

Por que está na sua mesa. A maior parte do risco e do custo no frontend está na *toolchain*, não no app: centenas de pacotes npm transitivos, plugins de build que executam na instalação e um bundler que você precisa manter atualizado. Nexa remove essa camada por completo. Isso é, ao mesmo tempo, um argumento de **segurança** (sem supply chain para atacar ou auditar) e de **arquitetura** (nada para rebuildar, re-lockar ou migrar).

0

dependências
de runtime

0

passos de build
no dev

0

eval / código
dinâmico

61

componentes
de UI

~300

testes x 3
engines

MIT

vendorizado, é
seu



Para Segurança

- **Sem supply chain.** Zero código de terceiros em runtime; nada de npm nem no runtime nem no build.
- **Compatível com CSP.** Sem eval/new Function — roda sob política estrita sem unsafe-eval.
- **Auditável.** O núcleo inteiro são poucos módulos legíveis que você pode vendorizar e diffar.
- **Seguro por padrão.** A saída do SSR é HTML-escapada; um `guard safeUrl()` já vem incluso.



Para Arquitetura

- **Sem build.** Dev é F5 em módulos ES puros; deploy é copiar arquivos estáticos.
- **Longevidade.** Sem bundler/transpiler para envelhecer; a plataforma é o runtime.
- **Baterias inclusas.** Componentes, design system, router, forms, SSR, add-ons — um só fornecedor.
- **Deploy em qualquer lugar.** Host estático, Python ou nginx; bundle single-file opcional.

A versão em uma frase: a Nexa troca o ecossistema npm por um runtime zero-dependência e no-build — uma redução deliberada de superfície de ataque e de custo de manutenção no longo prazo, ao preço de um mercado menor de componentes de terceiros (veja os trade-offs francos na §07).

02 O argumento de segurança

Sua superfície de ataque é o que você entrega. A Nexa entrega o framework e nada mais.

Uma SPA típica puxa centenas de pacotes npm e um toolchain de build que **executa código na instalação** — cada um, uma porta de supply chain. A Nexa não tem nada disso.

DIMENSÃO	STACK SPA TÍPICA	NEXA
Dependências de runtime	dezenas-centenas (transitivas)	0
Dependências de build / dev	centenas (bundler, transpiler, plugins)	0 — sem build
Execução de código na instalação	sim (scripts de ciclo do npm)	nenhuma — sem instalação
Código dinâmico (eval)	varia por dependência	nenhum no framework
O que você audita	node_modules + churn de lockfile	poucos módulos legíveis que você vendoriza

Não acredite na nossa palavra — verifique

```
verificar · reproduzível a partir de um clone

$ cat package.json | grep '"dependencies"'
"dependencies": {} → 0 deps · nada para instalar
$ grep -rE "from ['\"](https?:|[a-z@])" dist/*.js
(no output) → 0 imports externos
$ grep -rE "\beval\(|new Function(\" dist/*.js
(no output) → roda sob CSP estrita
```

CSP estrita, de fábrica

Sem geração de código em runtime: uma política endurecida dispensa unsafe-eval. Handlers usam addEventListener, não atributos inline.

```
Content-Security-Policy:
  script-src 'self'; // sem
  'unsafe-eval'
```

Renderização segura por padrão

A saída do SSR é HTML-escapada. Um guard nativo neutraliza ataques de URL javascript:/data::

```
import { safeUrl } from
'/dist/nexa.js';
h('a', { href: safeUrl(user.site) },
'Visitar')
```

Distribuição & integridade. Entregue como arquivos vendorizados ou CDN fixada em uma release. Para a postura mais forte, fixe em um commit e adicione um hash de Subresource Integrity (integrity) — o navegador recusa qualquer arquivo que não bata.

03 O argumento de arquitetura

Sem build — o app é o artefato de deploy

Um app Nexa é um `index.html` e módulos ES. Desenvolver é “salvar & atualizar”; não há passo de compilação para esperar, depurar ou proteger. O que você escreve é o que roda e o que é entregue — source maps são desnecessários porque não há transformação.

```
<!doctype html>
<class="tok-f">link rel="stylesheet"
href="/dist/nexa-ui.css">
<class="tok-f">div id="app"></class="tok-f">div>
<class="tok-f">script type="module"
src="./app.js"></class="tok-f">script>
```

Essa é uma página Nexa completa, pronta para produção.

Modelo mental familiar

Quem é acostumado com React já é produtivo de cara: componentes são funções, estado e efeitos são hooks. O runtime é mais simples (avaliação eager, re-render de subárvore), mas a ergonomia é a que seu time já conhece.

```
import { h, useState, useEffect } from
'/dist/nexa.js';

function Counter() {
  const [n, setN] = useState(0);
  useEffect(() => { document.title =
`${n}`; }, [n]);
  return h('button', { onClick: () =>
setN(v => v + 1) },
    `Cliques: ${n}`);
}
```

Longevidade & baixo lock-in

Não há major de bundler para perseguir, nem esteira de migração de framework. A Nexa mira o próprio sistema de módulos do navegador, então um app escrito hoje continua rodando enquanto os módulos ES existirem. Se a própria Nexa parasse, você já tem todo o código MIT em mãos — ele não apodrece como uma árvore de dependências sem manutenção.

Baterias inclusas, um só fornecedor

Em vez de montar e proteger uma dúzia de bibliotecas, uma única árvore vendorizada cobre a superfície:

~60 componentes

Design system + temas

Router

Forms

SSR + hydration

i18n

a11y auditada

Apresentações

Animação

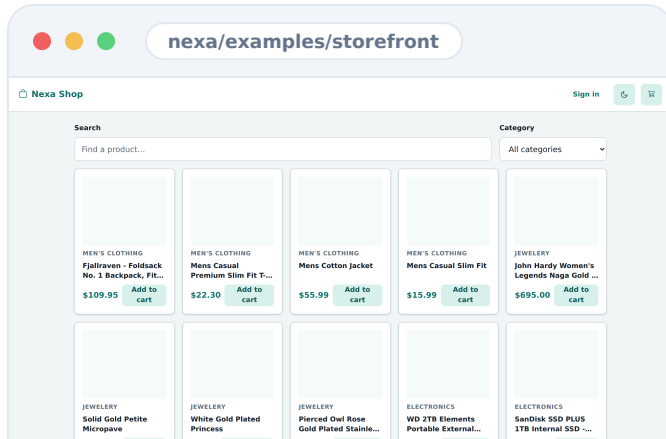
Diagramas

Editor de código

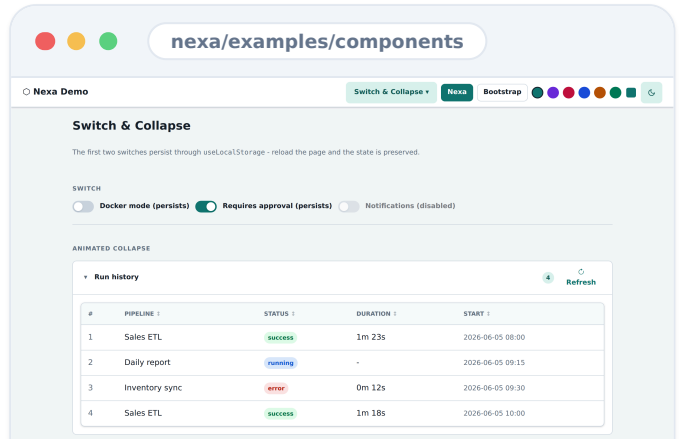
Encaixa em times Python ou com pouca infra. Todo o tooling da Nexa — dev server, runner de testes, validadores, bundler opcional — é biblioteca padrão do Python. Não há Node para instalar nas máquinas dos devs, no CI ou nos servidores. Deploy é arquivo estático atrás de qualquer host, um processo Python ou nginx.

04 Feito com Nexa

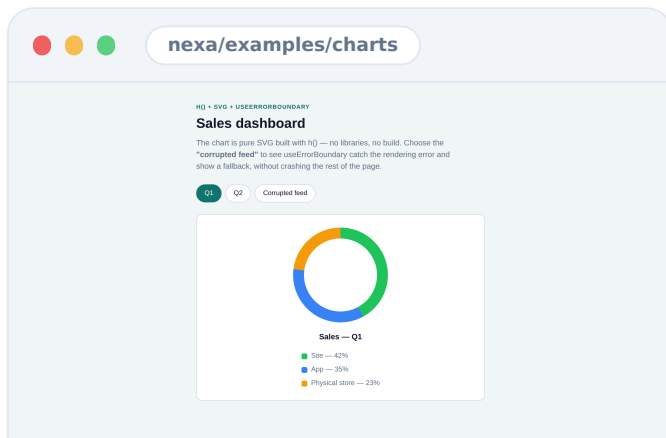
São apps de exemplo reais do repositório, capturados em um navegador headless. Todos são arquivos estáticos, sem build — os mesmos componentes, design tokens e add-ons que seu time usaria.



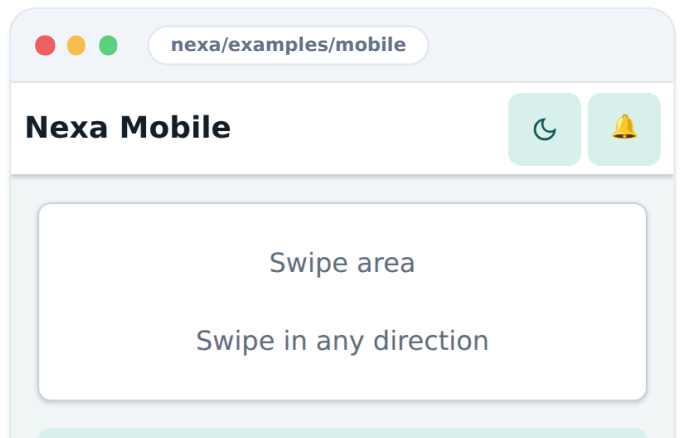
Loja e-commerce — grade de produtos, busca ao vivo, filtro por categoria, carrinho e dark mode, tudo da biblioteca de componentes.



Design system — switches, tabelas ordenáveis, collapse animado, além de troca de tema (Nexa/Bootstrap) e de paleta ao vivo.



Dashboard de dados — gráficos desenhados em SVG puro com `h()`, sem biblioteca de charts, com error boundary pegando um feed corrompido.



Shell mobile — app bar, bottom nav, FAB, hooks de gesto (swipe / long-press) e safe-area insets, grid mobile-first.

05 Experiência de desenvolvimento

Familiar para quem conhece React, mas mais simples por baixo. Alguns trechos mostram toda a superfície — sem JSX, sem build, módulos ES de verdade.

Um formulário validado

useForm nativo + um componente de campo:

```
const { field, handleSubmit, isSubmitting } = useForm({
  initialValues: { email: '' },
  validate: (v) => ({
    email: v.email.includes('@') ? '' :
    'Inválido',
  }),
  onSubmit: async (v) => { await
    api.login(v); },
});

h('form', { onSubmit: handleSubmit() },
  h(TextField, { ...field('email'),
    label: 'E-mail' }),
  h(Button, { type: 'submit', disabled:
    isSubmitting },
    'Entrar'));
```

Renderização no servidor (SSR)

Mesmos componentes; saída escapada; hidrata no cliente:

```
import { renderToString, hydrate }
  from '/dist/nexa-server.js';

// servidor: string HTML segura e
// escapada
const html = renderToString(App);

// cliente: adota o DOM do servidor no
// lugar
hydrate(App,
  document.getElementById('app'));
```

Temas por tokens

```
// dark mode + paleta, sem precisar de
// provider
const { toggleTheme } = useTheme();
const { setPalette } = usePalette();
```

Tudo na capa é essa mesma stack. A apresentação com zoom, o painel de sintetizador e a galeria de componentes são todos módulos simples como estes — os add-ons visuais (apresentações, animação, diagramas, editor de código) usam o mesmo `h()` e hooks, só importados quando você precisa.

06 Maturidade de engenharia

“Zero dependências” não pode significar “sem testes”. A Nexa é validada como uma plataforma deve ser — e, mantendo a filosofia intacta, inteiramente com Python + um navegador de verdade, sem Node.

~300 testes

Engine, hooks, componentes, SSR, ally, add-ons e segurança — asseridos contra um DOM real.

3 engines

A suíte roda em Chromium, Firefox e WebKit no CI a cada mudança.

Gates estáticos

Um validador checa imports resolvendo, assets existindo, ausência de monólitos, versões em sincronia e minificados batendo.

Integração contínua

- Validação estática + checagens de integridade de build (minificados/derivados não podem divergir).
- Suíte completa de browser nas três engines, sem fail-fast (todas as engines reportadas).
- Todos os jobs são python3 — o CI não precisa de acesso ao registry npm.

Superfície de QA documentada

- Um runbook de QA e um catálogo de cenários com IDs (~180 casos) cobrem cada hook, componente e add-on.
- Uma spec de geração por IA mantém o código gerado correto e idiomático.
- Acessibilidade é testada (focus traps, roving tabindex, ARIA), não presumida.

Auditabilidade como feature. O runtime central são alguns milhares de linhas de módulos ES legíveis e não-minificados. Um revisor de segurança consegue ler o framework inteiro, de ponta a ponta, em uma tarde — algo simplesmente impossível com uma árvore de dependências convencional.

07 Trade-offs francos

Um documento que só lista pontos fortes não sobrevive a uma revisão de arquitetura. Aqui está onde a Nexa te custa algo, e como esses custos são limitados.

✓ Onde a Nexa encaixa melhor

- Ferramentas internas, dashboards, painéis admin, apps de negócio.
- Ambientes onde risco de supply chain, air-gapping ou janelas longas de suporte importam.
- Times sem (ou querendo largar) um pipeline de build Node; times Python/com pouca infra.
- Projetos que valorizam auditabilidade e longevidade acima da amplitude do ecossistema.

△ Onde pensar duas vezes

- Você precisa de um componente grande e específico de terceiros React/Vue (mapas, grids ricos) — o mercado da Nexa é menor.
- Padrões da empresa exigem um framework específico ou um pool de contratação.
- Você depende de recursos de build-time (tree-shaking pesado de um app enorme, workflows TS-first) — a Nexa favorece simplicidade em runtime.

PREOCUPAÇÃO QUE UM REVISOR LEVANTARÁ

RESPOSTA FRANCA

“Projeto menor / mais novo, um só mantenedor.”

Verdade. Mitigado estruturalmente: é MIT e **vendorizado**, então o código é seu; com zero dependências não há árvore para apodrecer, e o runtime inteiro é auditável e forkável. Sua continuidade não depende do upstream seguir ativo.

“Sem ecossistema npm de componentes.”

Correto — essa é a troca deliberada. Os ~60 componentes nativos + add-ons cobrem a maior parte da UI; o resto é DOM puro/Web Components, que interoperam direto.

“Sem build de TypeScript.”

Traz declarações `.d.ts` escritas à mão para suporte no editor/tooling; você tem tipos sem passo de compilação, embora não um fluxo de autoria TS-first.

“No-build é realmente performático?”

Para a maioria dos apps, sim — HTTP/2 + uma CDN fixada cuidam do carregamento de módulos; um bundler **opcional** Python/esbuild produz um deploy single-file quando você quiser, ainda sem Node.

08 Deploy, integração & próximo passo

Opções de deploy

- **Host estático / CDN** — copie a pasta; fixe a CDN em uma tag + SRI.
- **Python ou nginx** — para SSR, roteamento SPA-fallback, ou gzip/brotli + cache imutável.
- **Bundle single-file** — opcional, para deploy em uma requisição; ainda com zero Node.
- **Air-gapped** — tudo é vendorizado; nada chama pra casa.

Integra com o que você já tem

- Agnóstico de backend — fala com qualquer API HTTP/JSON (seus serviços atuais).
- A entrada SSR roda em qualquer runtime de servidor para first-paint + SEO.
- Encaixa numa página existente (monta em um <div>); adotável de forma incremental.
- Compõe com Web Components nativos e DOM puro.

PRÓXIMO PASSO RECOMENDADO

Pilote em uma ferramenta interna. Escolha um dashboard ou tela admin de baixo risco, construa com Nexa, e passe pela sua revisão de segurança padrão e pelas checagens reproduzíveis da §02. Você avalia a coisa real — zero-install, CSP-limpo, auditável — contra a sua própria régua, sem precisar levantar toolchain nenhum antes.

VERIFIQUE

Reproduza cada alegação de segurança a partir de um clone — veja §02.

LEIA

O runtime completo é não-minificado, MIT e auditável de ponta a ponta.

TENHA

Vendorize. Sem registry, sem lockfile, sem passo de instalação no seu pipeline.

Nexa

Preparado para avaliação interna de segurança & arquitetura · Framework sob licença MIT · As alegações deste documento são reproduzíveis a partir do repositório-fonte.